MANUAL TECNICO

MAIN

El programa main es la entrada principal del sistema y contiene el menú principal que permite al usuario realizar diferentes acciones. El programa está estructurado en un bucle que se ejecuta hasta que el usuario decide salir.

- Funciones Principales

Inicio de Sesión: El usuario debe proporcionar un nombre de usuario y una contraseña. Si los datos coinciden con los predeterminados (en este caso, "admin" y "123"), se permite el acceso al menú principal.

Menú Principal: Este menú ofrece al usuario varias opciones, como cargar archivos, administrar sucursales, generar reportes y salir del programa.

Carga de Archivos JSON: El programa permite cargar archivos JSON que contienen información sobre sucursales, rutas entre sucursales y técnicos.

Administración de Sucursales: Esta opción permite al usuario realizar operaciones relacionadas con una sucursal específica, como cargar técnicos, generar recorridos óptimos, obtener información sobre un técnico en particular, listar técnicos y generar reportes.

- Funciones para Cargar Archivos JSON

leerJson(): Esta función carga un archivo JSON que contiene datos de sucursales. Lee los datos del archivo y los almacena en una estructura de árbol AVL.

leerJson2(): Carga un archivo JSON que contiene datos de rutas entre sucursales. Lee los datos y los almacena en un grafo.

leerJson3(): Carga un archivo JSON que contiene datos de técnicos. Lee los datos y los almacena en una tabla hash para facilitar la búsqueda y el acceso.

Arbol AVL (Sucursales)

Node_t: Define la estructura de un nodo del árbol AVL. Cada nodo contiene información sobre una sucursal, como su ID, departamento, dirección, contraseña y un contador de trabajos realizados. También tiene un enlace a su tabla hash asociada y punteros a sus nodos hijo izquierdo y derecho.

Tree_t: Define la estructura del árbol AVL, que contiene un puntero a la raíz del árbol.

- Funciones Principales

new_avl(self): Inicializa un árbol AVL vacío.

insert_node(tree, id, dir, dept, contra): Inserta un nuevo nodo (sucursal) en el árbol AVL.

- Rotaciones:

  rotationII: Rotación doble a la izquierda.

  rotationDD: Rotación doble a la derecha.

  rotationDI: Rotación doble izquierda-derecha.

  rotationID: Rotación doble derecha-izquierda.

  graph_avl_tree(self): Genera un archivo DOT y una imagen PNG que representa el árbol AVL.

  searchNode(tree, id, contra): Busca un nodo (sucursal) por su ID y contraseña.

  searchNodeESPE(tree, id, DPI): Busca un nodo (sucursal) por su ID y muestra información específica relacionada con un técnico (identificado por su DPI).

  searchNodeLIST(tree, id): Busca un nodo (sucursal) por su ID y muestra una lista de técnicos asociados.

  searchNodeTOP(tree, id): Busca un nodo (sucursal) por su ID y muestra los cinco técnicos con más trabajos.

  searchNode_hash(tree, id, tecnico): Busca un nodo (sucursal) por su ID y agrega una tabla hash (tecnico) asociada.

  printTopFiveMaxValues(tree): Imprime los cinco nodos con más trabajos.

  searchNode_tecnico_trabajo(tree, id, dpi): Busca un nodo (sucursal) por su ID y, si lo encuentra, incrementa el contador de trabajos y agrega un trabajo a un técnico específico (identificado por su DPI).

Estas funciones proporcionan una manera de gestionar eficazmente las sucursales, incluyendo la inserción, búsqueda y actualización de datos relacionados con las sucursales y los técnicos asociados.

RUTAS

El módulo routes.f90 proporciona herramientas para trabajar con grafos y realizar análisis de rutas en el lenguaje de programación Fortran. Este manual describe los tipos de datos y métodos disponibles en el módulo, así como su uso y ejemplos de implementación.

Nodo:

id: Identificador único de la arista.

weight: Peso de la arista, que representa la distancia entre nodos.

parent_id: ID del nodo padre.

printers: Número de impresoras a lo largo de la arista.

next: Puntero a la siguiente arista en la lista.

prev: Puntero a la arista anterior en la lista.

Lista de Nodos:

head: Puntero al primer elemento de la lista.

tail: Puntero al último elemento de la lista.

Métodos:

add_sorted: Añade una arista a la lista ordenada.

add_sorted_2: Añade una arista a la lista ordenada en sentido inverso.

pop: Elimina y devuelve la primera arista de la lista.

is_empty: Verifica si la lista está vacía.

merge: Fusiona dos listas de aristas.

merge_2: Fusiona dos listas de aristas en sentido inverso.

add_weight: Añade un peso a todas las aristas de la lista.

Nodo_Resultado:

```
type result
    integer :: id             ! Identificador del nodo
    integer :: weight         ! Peso acumulado del camino
    integer :: printers       ! Número de impresoras acumulado
    type(result), pointer :: next => null()   ! Puntero al siguiente resultado
end type result
```

TABLA HASH

El módulo hash_table proporciona una implementación de una tabla hash en Fortran. Este tipo de estructura de datos es útil para almacenar y recuperar rápidamente datos utilizando una función hash para calcular la ubicación de almacenamiento de cada elemento.

1. Tipos de Datos

1.1. Tipo tecnico

El tipo tecnico representa la estructura de datos para almacenar información de un técnico. Contiene los siguientes campos:

trabajoss: Número de trabajos realizados por el técnico.

key: DPI del técnico.

telefono: Número de teléfono del técnico.

nombre, apellido, direccion: Información personal del técnico.

1.2. Tipo HashTable

El tipo HashTable representa la tabla hash. Contiene los siguientes campos:

- elements: Número de elementos almacenados en la tabla.
- array: Array de elementos de tipo tecnico.
- Métodos para HashTable
- insert: Inserta un nuevo elemento en la tabla.
- print: Imprime todos los elementos de la tabla.
- search: Busca un elemento en la tabla.
- grafico: Genera un gráfico de la tabla.
- get_data: Obtiene los datos de un elemento en la tabla.
- print_top_five_techs: Imprime los cinco técnicos con más trabajos.
- get_data_trabajo: Obtiene los datos de un técnico y registra un nuevo trabajo.

2. Métodos de Manipulación de Datos

2.1. insert

Este método inserta un nuevo elemento en la tabla. Si la tabla está más del 70% llena, se realiza una reasignación de memoria.

subroutine insert(self, key, nombre, apellido, direccion, telefono)

2.2. print

Este método imprime todos los elementos de la tabla.

subroutine print(self)

2.3. search

Este método busca un elemento en la tabla utilizando su clave.

subroutine search(self, key)

2.4. grafico

Este método genera un gráfico de la tabla.

subroutine grafico(self, filename)

ARBOL MERKLE

1. Tipos de Datos

1.1. Tipo data

El tipo data representa los datos almacenados en el árbol de Merkle. Contiene los siguientes campos:

- uid: Identificador único del dato.
- id_origin: Identificador del origen.
- address_origin: Dirección del origen.
- id_destination: Identificador del destino.
- address_destination: Dirección del destino.
- cost_between: Costo entre el origen y el destino.
- hash_value: Valor del hash para los datos.

1.2. Tipo hash_node

El tipo hash_node representa un nodo en el árbol de Merkle. Contiene los siguientes campos:

- uid: Identificador único del nodo.
- hash: Valor del hash del nodo.
- left: Referencia al hijo izquierdo.
- right: Referencia al hijo derecho.
- dataref: Referencia a los datos asociados.

1.3. Tipo merkle

El tipo merkle representa el árbol de Merkle. Contiene los siguientes campos:

- top_hash: Referencia al nodo raíz del árbol.
- data_head: Referencia al primer dato.
- data_tail: Referencia al último dato.
- pos: Posición actual en la generación de hashes.
- Métodos para merkle
- add_data: Agrega datos al árbol.
- get_data: Obtiene datos del árbol.
- data_length: Obtiene la longitud de los datos.
- create_tree: Crea el árbol de Merkle.

- gen_hash: Genera los hashes para el árbol.
- generate: Genera el árbol de Merkle.
- merkle_dot: Genera un archivo DOT para visualizar el árbol.
- merkle_dot_rec: Método recursivo para generar el archivo DOT.